Entrega

Actividad Sumativa

Reutilización de *software*

Módulo: Temas avanzados en diseño de *software*

Unidad 2

Nota

Tenga en cuenta que el tutor le indicará qué herramienta requiere y qué estrategia deberá desarrollar para evidenciar su participación individual en un trabajo colaborativo.

Indicadores de logro

Realiza modelos de *software* como parte de la etapa de diseño para la construcción de un sistema computacional.

Instrucciones para realizar la entrega

Se deben reunir en grupos de trabajo y analizar el problema que se plantea (definido en la descripción de la actividad). Las actividades a desarrollar son:

- Una vez se ha leído y entendido el problema planteado, deben proceder a realizar el modelo de datos (clases) en lenguaje UML2.5, que satisface el problema planteado y su solución.
- Luego, para diligenciar la tabla 1 que encuentra disponible en el **material de trabajo** de la unidad, se debe definir cuáles son los posibles componentes que se pueden extraer para cada área de negocio e indicar cuáles son las funcionalidades que expone cada uno (ver

el video de la unidad, que muestra cómo se pueden extraer componentes a partir de los conceptos de negocio planteados en el diagrama de clases).

- Posteriormente, debe agregar en la tabla 1 a los componentes y a las funcionalidades que se determinan para cada uno.

Finalmente, se debe construir un modelo de componentes (de acuerdo con la lectura fundamental de la unidad 2) donde se muestre cómo se relacionan entre sí para proveer las funcionalidades de la organización.

Descripción de la actividad

En la empresa PoliMarket, se cuenta con diferentes áreas de negocio, cada una con sistemas de información separados, como son: bodega, ventas, recursos humanos, proveedores y entregas.

Los vendedores deben ser autorizados por **recursos humanos** para que puedan interactuar con los demás sistemas de la organización. Los vendedores se conectan con ventas para realizar las correspondientes acciones de las tareas asignadas, donde cuentan con un listado de los posibles clientes y de los productos disponibles. Asimismo, ventas se conecta con la bodega para poder ver la disponibilidad de los productos. De la misma manera, bodega se conecta con proveedores para poder tener un registro de los productos adquiridos por la empresa y saber a qué empresa solicitar más cuando se terminen las existencias en la bodega. Finalmente, entregas se comunica con ventas para saber qué productos debe entregar y a quién, y también se comunica con bodega para registrar la salida de productos del *stock*.

Condiciones de la entrega

Se deben seguir los criterios de citación y referenciación académica, de acuerdo con las normas APA.

El trabajo debe ser realizado en los grupos definidos por el tutor. Consta de cinco elementos:

1. Diagrama de datos (clases) como una imagen en formato BMP y utilizando la sintaxis vista para estos diagramas en el tema 1 (uso adecuado de las relaciones y estas deben indicar la multiplicidad). Además, se debe usar la notación de UML 2.5.

   Tabla de definición de los componentes (mínimo dos por cada requisito seleccionado —capa de negocio y cliente— y diagrama de componentes UML, como una imagen en formato BMP, de acuerdo con la sintaxis vista en la lectura del tema 1 (definición de los componentes y cómo se relacionan entre sí).

2. Construcción del sistema en un lenguaje de desarrollo conversado previamente en clase. Los componentes deben mostrar la implementación de 5 requisitos funcionales extraídos de la descripción del problema. Debe estar acompañado de la construcción de por lo menos 2 clientes en diferentes plataformas – lenguajes. Cada uno de esos clientes debe consumir como mínimo 2 funcionalidades de los componentes construidos. Ver material de trabajo.

3. Conclusiones.

El trabajo debe ser entregado en la plataforma en la fecha indicada.

REUTILIZACIÓN DE SOFTWARE — POLIMARKET

Modelo de clases, componentes e implementación en Node.js

Presentado por: Alejandro De Mendoza
Politécnico Grancolombiano — Maestría en Arquitectura de Software
Temas Avanzados en Diseño de Software
Docente: Ing. Lina María Montoya Suárez
Bogotá D.C. — Marzo 2026

Introducción

El presente documento desarrolla la actividad sumativa de la Unidad 2 del módulo Temas Avanzados en Diseño de Software, centrada en la reutilización de software mediante el uso de componentes. A partir del caso de negocio propuesto para la empresa PoliMarket, se construye un modelo de clases UML, se identifican y definen los componentes por área de negocio, se elabora el diagrama de componentes y se implementan cinco requisitos funcionales en Node.js, acompañados de dos clientes en plataformas diferentes.

La arquitectura de componentes propuesta promueve principios como el bajo acoplamiento, la alta cohesión y la reutilización de lógica de negocio, permitiendo que cada área del sistema (Recursos Humanos, Ventas, Bodega, Proveedores y Entregas) opere de forma independiente e interoperable.

Parte 1. Diagrama de clases UML

A continuación se presentan las clases identificadas para el sistema PoliMarket, junto con sus atributos, métodos y justificación de inclusión. El diagrama fue generado con PlantUML usando la sintaxis UML 2.5.

Clases identificadas y justificación

Vendedor

Representa al empleado que realiza ventas. Requiere autorización de Recursos Humanos para interactuar con los demás sistemas. Atributos: id, nombre, email, autorizado. Métodos: solicitarAutorizacion(), realizarVenta(), consultarClientes(), consultarProductos().

RecursosHumanos

Gestiona la autorización de vendedores para acceder al sistema. Sin este componente no existe control de acceso. Atributos: id, responsable. Métodos: autorizarVendedor(), revocarAutorizacion(), listarVendedores().

Cliente

Persona que realiza compras en PoliMarket. Atributos: id, nombre, email, direccion. Métodos: getDatos(), getHistorialCompras().

Producto

Ítem disponible para la venta. Su stock es gestionado por la bodega. Atributos: id, nombre, precio, stock. Métodos: getDisponibilidad(), actualizarStock().

Venta

Representa una transacción comercial. Vincula al vendedor con el cliente y los productos adquiridos. Atributos: id, fecha, total, estado. Métodos: registrarVenta(), calcularTotal(), getDetalle().

Bodega

Gestiona el inventario físico. Interactúa con ventas, proveedores y entregas para controlar el flujo de productos. Atributos: id, ubicacion, capacidad. Métodos: verificarDisponibilidad(), registrarEntrada(), registrarSalida(), solicitarReabastecimiento().

Proveedor

Empresa externa que suministra productos a PoliMarket. Atributos: id, nombre, contacto, telefono. Métodos: registrarProducto(), getProductosSuministrados(), solicitarPedido().

Entrega

Gestiona la distribución de productos vendidos hacia los clientes. Se comunica con ventas para conocer qué entregar y con bodega para registrar la salida. Atributos: id, fecha, estado, direccionDestino. Métodos: registrarEntrega(), consultarPedidosPendientes(), confirmarEntrega(), registrarSalidaBodega().

Relaciones del modelo

- RecursosHumanos autoriza Vendedor — asociación 1 a muchos.
- Vendedor realiza Venta — asociación 1 a muchos.
- Venta pertenece a Cliente — asociación muchos a 1.
- Venta incluye Producto — asociación muchos a muchos.
- Venta consulta Bodega — dependencia (verificar disponibilidad).
- Bodega registra Proveedor — asociación (registro de productos suministrados).
- Entrega consulta Venta — dependencia (saber qué entregar).
- Entrega registra salida en Bodega — dependencia.

Código PlantUML — Diagrama de clases

El siguiente código debe ejecutarse en plantuml.com para generar el diagrama. Exportar en formato BMP según lo solicitado.

```
@startuml PoliMarket_Clases
skinparam classAttributeIconSize 0
skinparam classFontSize 13
skinparam classBackgroundColor #FAFAFA
skinparam classBorderColor #555555
skinparam arrowColor #333333
skinparam shadowing false
skinparam linetype ortho

class RecursosHumanos {
  - id : String
  - responsable : String
  + autorizarVendedor(vendedorId: String) : boolean
  + revocarAutorizacion(vendedorId: String) : void
  + listarVendedores() : List<Vendedor>
}

class Vendedor {
  - id : String
  - nombre : String
  - email : String
  - autorizado : boolean
  + solicitarAutorizacion() : void
  + realizarVenta(clienteId: String) : Venta
  + consultarClientes() : List<Cliente>
  + consultarProductos() : List<Producto>
}
```

```
class Cliente {
  - id : String
  - nombre : String
  - email : String
  - direccion : String
  + getDatos() : Map<String, String>
  + getHistorialCompras() : List<Venta>
}

class Producto {
  - id : String
  - nombre : String
  - precio : double
  - stock : int
  + getDisponibilidad() : int
  + actualizarStock(cantidad: int) : void
}

class Venta {
  - id : String
  - fecha : Date
  - total : double
  - estado : String
  + registrarVenta() : void
  + calcularTotal() : double
  + getDetalle() : Map<String, Object>
}

class Bodega {
  - id : String
  - ubicacion : String
  - capacidad : int
  + verificarDisponibilidad(productoId: String) : int
  + registrarEntrada(productoId: String, cantidad: int) : void
  + registrarSalida(productoId: String, cantidad: int) : void
  + solicitarReabastecimiento(productoId: String) : void
}

class Proveedor {
  - id : String
  - nombre : String
  - contacto : String
  - telefono : String
  + registrarProducto(producto: Producto) : void
  + getProductosSuministrados() : List<Producto>
  + solicitarPedido(productoId: String, cantidad: int) : void
}

class Entrega {
  - id : String
  - fecha : Date
  - estado : String
  - direccionDestino : String
  + registrarEntrega() : void
  + consultarPedidosPendientes() : List<Venta>
  + confirmarEntrega(entregaId: String) : void
  + registrarSalidaBodega(productoId: String, cantidad: int) : void
}

' RELACIONES
RecursosHumanos "1" --> "0..*" Vendedor : autoriza >
Vendedor "1" --> "0..*" Venta : realiza >
```

```
Venta "0..*" --> "1" Cliente : pertenece a >
Venta "0..*" --> "1..*" Producto : incluye >
Venta ..> Bodega : <<consulta>>
Bodega "1" --> "1..*" Proveedor : registra >
Entrega ..> Venta : <<consulta>>
Entrega ..> Bodega : <<registra salida>>
@enduml
```

Figura 1. Diagrama de clases UML — PoliMarket (insertar imagen BMP aquí)

[Espacio para insertar Figura 1: imagen BMP exportada de plantuml.com]

Parte 2. Tabla de definición de componentes

A continuación se presenta la Tabla 1 con los componentes identificados para cada área de negocio de PoliMarket. Se definieron mínimo dos componentes por área, especificando las funcionalidades que cada uno expone.

Tabla 1. Definición de componentes por área de negocio

Área de negocio	Componente	Funcionalidades que expone
Recursos Humanos	ComponenteAutorizacion	autorizarVendedor(), revocarAutorizacion()
Recursos Humanos	ComponenteGestionVendedores	listarVendedores(), consultarEstadoVendedor()
Ventas	ComponenteRegistroVentas	registrarVenta(), calcularTotal(), getDetalle()
Ventas	ComponenteConsultaVentas	listarClientes(), listarProductosDisponibles()
Bodega	ComponenteInventario	verificarDisponibilidad(), registrarEntrada(), registrarSalida()
Bodega	ComponenteReabastecimiento	solicitarReabastecimiento(), consultarProveedores()
Proveedores	ComponenteGestionProveedores	registrarProducto(), getProductosSuministrados()
Proveedores	ComponentePedidos	solicitarPedido(), consultarEstadoPedido()
Entregas	ComponenteGestionEntregas	registrarEntrega(), confirmarEntrega()
Entregas	ComponenteLogistica	consultarPedidosPendientes(), registrarSalidaBodega()

Fuente: elaboración propia.

Parte 3. Diagrama de componentes UML

El diagrama de componentes representa la arquitectura del sistema PoliMarket, mostrando cómo los componentes de cada área de negocio se relacionan entre sí para proveer las funcionalidades de la

organización.

Código PlantUML — Diagrama de componentes

Ejecutar en plantuml.com y exportar en formato BMP.

```
@startuml PoliMarket_Componentes
skinparam componentStyle uml2
skinparam backgroundColor #FFFFFF
skinparam componentBackgroundColor #FAFAFA
skinparam componentBorderColor #555555
skinparam arrowColor #333333
skinparam shadowing false
skinparam linetype ortho

package "Recursos Humanos" {
  [ComponenteAutorizacion] as CA
  [ComponenteGestionVendedores] as CGV
  CA ..> CGV : usa
}

package "Ventas" {
  [ComponenteRegistroVentas] as CRV
  [ComponenteConsultaVentas] as CCV
  CRV ..> CCV : usa
}

package "Bodega" {
  [ComponenteInventario] as CI
  [ComponenteReabastecimiento] as CR
  CI ..> CR : notifica
}

package "Proveedores" {
  [ComponenteGestionProveedores] as CGP
  [ComponentePedidos] as CP
  CGP ..> CP : genera
}

package "Entregas" {
  [ComponenteGestionEntregas] as CGE
  [ComponenteLogistica] as CL
  CGE ..> CL : coordina
}

CA --> CRV : <<autoriza acceso>>
CCV --> CI : <<consulta disponibilidad>>
CRV --> CGE : <<notifica pedido>>
CR --> CP : <<solicita pedido>>
CL --> CCV : <<consulta pedidos pendientes>>
CL --> CI : <<registra salida>>
CGP --> CI : <<registra entrada>>
@enduml
```

Figura 2. Diagrama de componentes UML — PoliMarket (insertar imagen BMP aquí)

[Espacio para insertar Figura 2: imagen BMP exportada de plantuml.com]

Parte 4. Implementación en Node.js

A continuación se presenta la implementación de los cinco requisitos funcionales extraídos del problema de

PoliMarket, desarrollada en Node.js con una arquitectura basada en componentes. El proyecto incluye dos clientes en plataformas diferentes.

Estructura del proyecto

- db.js — Base de datos en memoria (vendedores, clientes, productos, proveedores, ventas, entregas)
- components/ComponenteAutorizacion.js — RF1
- components/ComponenteRegistroVentas.js — RF2
- components/ComponenteInventario.js — RF3
- components/ComponenteReabastecimiento.js — RF4
- components/ComponenteGestionEntregas.js — RF5
- clients/web/servidor.js — Cliente 1: API REST con Express (puerto 3000)
- clients/cli/cliente-cli.js — Cliente 2: CLI interactivo por terminal

RF1 — Autorizar vendedor (Recursos Humanos)

RRHH habilita a un vendedor para que pueda interactuar con los demás sistemas. Sin esta autorización, el vendedor no puede registrar ventas. Implementado en ComponenteAutorizacion.js.

```
autorizarVendedor(vendedorId) {
  const vendedor = db.vendedores.find(v => v.id === vendedorId);
  if (!vendedor) return { ok: false, mensaje: 'Vendedor no encontrado.' };
  vendedor.autorizado = true;
  return { ok: true, mensaje: `Vendedor ${vendedor.nombre} autorizado.`, vendedor };
}
```

RF2 — Registrar venta (Ventas)

El vendedor registra una venta vinculando un cliente y uno o varios productos. Antes de proceder, verifica que el vendedor esté autorizado y que los productos tengan stock disponible. Implementado en ComponenteRegistroVentas.js.

```
registrarVenta(vendedorId, clienteId, productosIds) {
  if (!ComponenteAutorizacion.estaAutorizado(vendedorId))
    return { ok: false, mensaje: 'Vendedor no autorizado.' };
  const venta = { id, fecha, vendedorId, clienteId, productos, total, estado };
  db.ventas.push(venta);
  return { ok: true, mensaje: 'Venta registrada.', venta };
}
```

RF3 — Verificar disponibilidad en bodega (Bodega)

Antes de confirmar una venta, el sistema consulta la disponibilidad del producto en bodega. Si el stock es cero, la venta no procede. Implementado en ComponenteInventario.js.

```
verificarDisponibilidad(productoId) {
  const producto = db.productos.find(p => p.id === productoId);
  if (producto.stock === 0)
    return { ok: false, mensaje: 'Sin stock disponible.' };
  return { ok: true, mensaje: `Stock: ${producto.stock}`, producto };
}
```

RF4 — Solicitar reabastecimiento (Bodega → Proveedor)

Cuando el stock de un producto cae por debajo del mínimo definido (3 unidades), la bodega solicita reabastecimiento al proveedor correspondiente. Implementado en ComponenteReabastecimiento.js.

```
solicitarReabastecimiento(productoId, proveedorId, cantidad) {
  if (producto.stock >= STOCK_MINIMO)
    return { ok: false, mensaje: 'Stock suficiente.' };
  const pedido = { id, productoId, proveedorId, cantidad, estado: 'solicitado' };
  ComponenteInventario.registrarEntrada(productoId, cantidad);
  return { ok: true, mensaje: 'Reabastecimiento solicitado.', pedido };
}
```

RF5 — Registrar entrega (Entregas)

El área de entregas consulta las ventas pendientes, registra la entrega al cliente y actualiza el estado de la venta. Implementado en ComponenteGestionEntregas.js.

```
registrarEntrega(ventaId) {
  const entrega = { id, ventaId, clienteId, direccionDestino, productos, estado };
  db.entregas.push(entrega);
  venta.estado = 'entregado';
  return { ok: true, mensaje: 'Entrega registrada.', entrega };
}
```

Clientes implementados

Cliente 1 — API REST con Express (plataforma web)

Expone todos los componentes como endpoints HTTP. Permite consumir los cinco RF a través de peticiones POST y GET. Se ejecuta en el puerto 3000 e incluye una interfaz web en http://localhost:3000.

```
POST /rrhh/autorizar       — RF1: Autorizar vendedor
POST /ventas/registrar     — RF2: Registrar venta
GET  /bodega/disponibilidad/:id — RF3: Verificar disponibilidad
POST /bodega/reabastecer   — RF4: Solicitar reabastecimiento
POST /entregas/registrar   — RF5: Registrar entrega
```

Cliente 2 — CLI interactivo (plataforma terminal)

Interfaz de línea de comandos que permite al usuario interactuar con el sistema a través de un menú numerado. Consume los cinco RF de forma interactiva sin necesidad de un servidor HTTP.

```
1. Autorizar vendedor          (RF1 - RRHH)
2. Registrar venta             (RF2 - Ventas)
3. Ver disponibilidad bodega   (RF3 - Bodega)
4. Solicitar reabastecimiento  (RF4 - Bodega)
5. Registrar entrega           (RF5 - Entregas)
```

Instrucciones de ejecución

- Descomprimir el archivo polimarket.zip
- Ejecutar: npm install
- Cliente web: npm run start:web — acceder desde http://localhost:3000
- Cliente CLI: cd polimarket && npm run start:cli

Conclusiones

El desarrollo de esta actividad permitió aplicar de manera práctica los conceptos de arquitectura de componentes y reutilización de software en el contexto de un sistema de información empresarial. A partir del análisis del caso PoliMarket, fue posible identificar con claridad las entidades del negocio, modelarlas mediante un diagrama de clases UML y derivar a partir de ellas un conjunto de componentes reutilizables organizados por área de negocio.

La definición de componentes con responsabilidades bien delimitadas, como ComponenteAutorizacion, ComponenteInventario o ComponenteGestionEntregas, evidencia la aplicación de principios fundamentales del diseño de software como la separación de responsabilidades, el bajo acoplamiento y la alta cohesión. Cada componente expone únicamente las funcionalidades necesarias, lo que facilita su mantenimiento, prueba y reutilización en otros contextos del sistema.

La implementación en Node.js demostró que una arquitectura basada en componentes permite construir clientes en plataformas completamente diferentes —una API REST y una interfaz de línea de comandos— compartiendo la misma lógica de negocio sin duplicación de código. Esto valida el valor de la reutilización como principio arquitectónico: los cinco requisitos funcionales están implementados una sola vez en los componentes y son consumidos por ambos clientes de forma independiente.

Finalmente, el uso de herramientas como PlantUML para la generación de diagramas y Node.js para la implementación permitió construir una solución coherente, documentada y ejecutable, que refleja la importancia del modelado previo como base para el desarrollo de software de calidad.

Bibliografía

Booch, G., Rumbaugh, J., & Jacobson, I. (2005). El lenguaje unificado de modelado (UML): Guía del usuario (2.a ed.). Pearson Educación.

Pressman, R. S., & Maxim, B. R. (2020). Ingeniería de software: Un enfoque práctico (9.a ed.). McGraw-Hill.

Sommerville, I. (2016). Ingeniería de software (10.a ed.). Pearson Educación.

Szyperski, C. (2002). Component Software: Beyond Object-Oriented Programming (2nd ed.). Addison-Wesley.

Object Management Group (OMG). (2017). Unified Modeling Language (UML) Specification. https://www.omg.org/spec/UML/

PlantUML. (2024). PlantUML Language Reference Guide. https://plantuml.com/

OpenJS Foundation. (2024). Node.js Documentation. https://nodejs.org/en/docs/

Apreciado estudiante, haz tus entregas como DAVID EL RINOCERONTE, quien con transparencia y calidad, presenta siempre sus trabajos puntual.

Antes de subir tus archivos a la plataforma, lee detenidamente las siguientes indicaciones y minimiza inconvenientes:

1. Por tu seguridad y la seguridad de tus compañeros, nunca incluyas datos personales.

2. Antes de la fecha de cierre, podrás guardar el archivo en caso que sea necesario borrarlo o cambiarlo. Si lo envías para calificar, no podrás realizar ningún cambio sobre el mismo.

3. Cuando hayas cargado tu trabajo correctamente, podrás ver el nombre del archivo al lado derecho de tu pantalla.

4. Recuerda: debes cargar el archivo por lo menos dos horas antes de la hora de cierre. Evita problemas de saturación en el servidor.

5. Asegúrate de tener buena conexión a internet, cierra cualquier programa que pueda consumir el ancho de banda y no utilices internet móvil.

6. El uso de teléfonos inteligentes o tabletas está permitido únicamente para la visualización de tu material. No la uses para la presentación de actividades evaluativas, ni para cargar archivos de entregas grupales.

7. Si presentas algún inconveniente de la plataforma al momento de cargar el archivo, te recomendamos crear un caso adjuntando imagenes de soporte, donde se evidencie nombre de la actividad y/o URL respectiva, el error, la fecha y hora en que ocurrió.

¡Confiamos en que sigas, paso a paso, en el camino hacia la excelencia académica!

¿Das tu palabra de que realizarás esta actividad asumiendo de corazón nuestro

PACTO DE HONOR?